Bayesian Hyperparameter Optimisation

Wigner Summer Camp
Data and Compute Intensive Sciences Research Group

Balázs, Paszkál, Vince, Levente, Bence, Antal
Éva, Hajni

6–10 July 2026

Contents

Outline

The Hyperparameter Tuning Problem

- A **hyperparameter** is a configuration value set before training (e.g. regularisation strength, number of trees, learning rate).
- We seek the vector θ^* that maximises the cross-validation score $f(\theta)$:

$$\theta^* = \arg\max_{\theta \in \Theta} f(\theta). \tag{1}$$

- Evaluating $f(\theta)$ is expensive: it requires training and cross-validating the model for every candidate θ.

Search Strategies Compared

- Three standard strategies differ in whether they exploit past evaluations.

Strategy	**Uses past info?**	**Cost scaling**
Grid search	No	Exponential in dims.
Random search	No	Linear in dims.
Bayesian opt.	Yes	Sub-linear in dims.

- Bayesian optimisation reuses information from all past evaluations to focus the search on the most promising regions of Θ.

The Bayesian Approach

- A **surrogate model** (e.g. a Gaussian Process) builds a probabilistic approximation $\hat{f}(\theta)$ of the objective from all previously evaluated points.
- An **acquisition function** $\alpha(\theta)$ quantifies how promising a candidate θ is as the next evaluation point.
- At each iteration the next candidate is chosen as:

$$\theta_{n+1} = \arg\max_{\theta \in \Theta} \alpha(\theta; \{(\theta_i, f(\theta_i))\}_{i=1}^{n}) . \tag{2}$$

- This balances **exploration** (uncertain regions) and **exploitation** (known good regions).

Outline

Function Signature

`BayesSearchCV` is a scikit-learn compatible wrapper for Bayesian hyperparameter search, documented at scikit-optimize.github.io.

```
from skopt import BayesSearchCV

opt = BayesSearchCV(
    estimator,              # estimator to tune
    search_spaces,          # dict: param name -> search space
    n_iter=50,              # number of Bayesian evaluations
    cv=5,                   # cross-validation
    scoring=None,           # evaluation metric
    n_jobs=1,               # used cores (-1 = all)
    random_state=None,      # seed
    refit=True              # refit best model on all training data
)
```

After-Fit Attributes

After calling `opt.fit(X_train, y_train)`, the following attributes are available:

```
opt.best_params_     # best hyperparameter combination
opt.best_score_      # best mean cross-validation score
opt.best_estimator_  # estimator refitted on all training data
opt.cv_results_      # full history of all evaluations
```

- `best_estimator_` is ready to use for prediction on new data.
- `cv_results_['params']` and `cv_results_['mean_test_score']` list every configuration tried together with its score.

The `Real` and `Integer` Search Spaces

```python
from skopt.space import Real, Integer

# Continuous parameter on a linear scale
Real(low=0.01, high=10.0, prior='uniform')

# Continuous parameter on a logarithmic scale
Real(low=1e-4, high=1e2, prior='log-uniform')

# Integer parameter (e.g., number of trees)
Integer(low=10, high=500)
```

- Each space object is defined by its lower bound, upper bound, and sampling distribution (`prior`).

The Categorical Space

```
from skopt.space import Categorical

# Unordered discrete choices
Categorical(categories=['rbf', 'linear',
    'poly'])

# A typical search_spaces dictionary combining
    all types:
search_spaces = {
    'kernel': Categorical(['rbf', 'linear']),
    'C':      Real(1e-2, 1e2,
        prior='log-uniform'),
    'n':      Integer(10, 200)
}
```

- Each key in `search_spaces` maps a parameter name to its search space object.
- `Categorical` is used for unordered discrete choices such as kernel types or activation functions.

The `log-uniform` Prior

- With `prior='uniform'`, samples are spread evenly on a linear scale across $[a, b]$.
- For wide ranges this concentrates most samples near the upper bound, leaving the lower end undersampled.
- With `prior='log-uniform'`, the logarithm of the parameter is uniform:

$$\log \theta \sim \text{Uniform}(\log a,\ \log b). \tag{3}$$

- This assigns *equal probability to every order of magnitude.*

When to Use `log-uniform`

Example 1 (`Real(1e-2, 1e2, prior='log-uniform')`)

Equal probability is assigned to $[10^{-2}, 10^{-1}]$, $[10^{-1}, 10^{0}]$, $[10^{0}, 10^{1}]$, *and* $[10^{1}, 10^{2}]$.

- Use `'log-uniform'` for parameters where relative differences matter more than absolute ones: regularisation strengths (C, λ), learning rates (η), and kernel widths (γ).
- Use `'uniform'` (the default) for parameters where the effect is linear in scale, such as mixture weights or proportions.

`n_iter` and `n_jobs`

- `n_iter`: number of hyperparameter configurations to evaluate.
  - Total model fits required: $n\_\text{iter} \times k_{\text{cv}}$.
  - More iterations yield a better optimum but increase wall-clock time.
  - A reasonable starting point for moderate search spaces is 30–50.
- `n_jobs`: number of parallel jobs to run.
  - Set to -1 to use all available CPU cores.
  - Can significantly reduce wall-clock time for large `n_iter` or expensive estimators.

`cv`, `scoring`, and `random_state`

- `cv`: cross-validation strategy.
  - An integer k applies stratified k-fold CV (classification) or k-fold (regression).
  - A sklearn splitter (e.g. `StratifiedKFold`) gives finer control.
- `scoring`: the metric used to rank configurations.
  - Classification: `'accuracy'`, `'f1'`, `'roc_auc'`.
  - Regression: `'r2'`, `'neg_mean_squared_error'`.
  - `None` uses the estimator's default `score()` method.
- `random_state`: integer seed that ensures reproducible runs.

Outline

Setup: Imports and Data

We optimise the regularisation strength C of `LogisticRegression` on the breast cancer dataset. Distance-based models such as `LogisticRegression` require scaled features to converge reliably.

```
from skopt import BayesSearchCV
from skopt.space import Real
from sklearn.linear_model import
    LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_breast_cancer

X, y = load_breast_cancer(return_X_y=True)
```

Creating and Fitting the Search

```
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('clf',
        LogisticRegression(max_iter=1000))
])
opt = BayesSearchCV(
    estimator=pipe,
    search_spaces={'clf__C': Real(1e-3, 1e2,
        prior='log-uniform')},
    n_iter=20, cv=5, scoring='accuracy',
        random_state=42
)
opt.fit(X, y)
```

- The key `'clf__C'` follows the sklearn Pipeline convention: `step_name__param_name`.
- The objective function is $f(C) =$ mean 5-fold CV accuracy of the pipeline(C).

Accessing the Best Result

```
print(f"Best␣C:␣␣␣␣␣␣
    {opt.best_params_['clf__C']:.4f}")
print(f"Best␣score:␣{opt.best_score_:.4f}")
# Example output:
# Best C:      0.3142
# Best score: 0.9754

# Use the best pipeline for prediction on new
    data
y_pred = opt.best_estimator_.predict(X_new)
```

- `best_estimator_` is the full `Pipeline` (scaler + classifier) already refitted on the full training set with the optimal C.

Inspecting All Evaluated Configurations

```
for params, score in zip(
        opt.cv_results_['params'],
        opt.cv_results_['mean_test_score']):
    print(params, f"-> {score:.4f}")
# Example output:
# {'clf__C': 0.0132} -> 0.9173
# {'clf__C': 4.5871} -> 0.9736
# {'clf__C': 0.3142} -> 0.9754
```

- Iterating `cv_results_` is useful for plotting the optimisation history and checking convergence.

Outline

Multi-Parameter Search: Problem Setup

- We tune three hyperparameters of an `SVC` simultaneously: C, γ, and the kernel type.

```
from skopt import BayesSearchCV
from skopt.space import Real, Categorical
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_breast_cancer

X, y = load_breast_cancer(return_X_y=True)
```

Multi-Parameter Search: The Call

```
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('svc',    SVC())
])
opt = BayesSearchCV(
    estimator=pipe,
    search_spaces={
        'svc__C':      Real(1e-2, 1e2,
            prior='log-uniform'),
        'svc__gamma':  Real(1e-4, 1e-1,
            prior='log-uniform'),
        'svc__kernel': Categorical(['rbf',
            'poly'])
    },
    n_iter=40, cv=5, scoring='accuracy',
        random_state=42
)
```

Multi-Parameter Search: Fitting and Results

```
opt.fit(X, y)
print(opt.best_params_)
print(f"Best␣score:␣{opt.best_score_:.4f}")
# Example output:
# {'svc__C': 12.4, 'svc__gamma': 0.0012,
    'svc__kernel': 'rbf'}
# Best score: 0.9894
```

- The objective function is $f(C, \gamma, \text{kernel}) =$ mean CV accuracy of the pipeline$(C, \gamma, \text{kernel})$.

Why Use Bayesian Search?

- The surrogate model captures correlations between parameters automatically, so the search focuses on jointly promising regions.
- **Grid search:** a $10 \times 10 \times 2$ grid requires 200 evaluations; Bayesian search finds a comparable optimum in 40.
- The advantage grows with the number of hyperparameters: grid search cost scales exponentially while Bayesian search scales much more slowly.
- `log-uniform` priors are used for both C and γ since their effects are relative, spanning several orders of magnitude.

Grid vs Bayesian: Scaling

- The table shows evaluations required by each method, using 10 grid points per parameter.

Parameters	**Grid (10 pts each)**	**Bayesian**
2	$10^2 = 100$	~ 30
4	$10^4 = 10\,000$	~ 50
6	$10^6 = 1\,000\,000$	~ 100

Outline

Exercise 1 – Identify the Search Space Type

For each hyperparameter below, choose the most appropriate search space object (`Real`, `Integer`, or `Categorical`) and decide whether `prior='log-uniform'` should be used.

(a) SVM regularisation strength $C \in [10^{-3}, 10^{3}]$.

(b) Number of trees in a Random Forest, from 10 to 500.

(c) Neural network activation function: `'relu'`, `'tanh'`, or `'sigmoid'`.

(d) Gradient boosting learning rate $\eta \in [10^{-5}, 10^{-1}]$.

(e) Maximum depth of a decision tree, from 2 to 30.

Exercise 1 – Solution (1/2)

```
# (a) Continuous, spans 6 orders of magnitude
    -> log-uniform.
Real(1e-3, 1e3, prior='log-uniform')

# (b) Discrete count -> Integer (uniform prior).
Integer(10, 500)

# (c) Unordered discrete choices -> Categorical.
Categorical(['relu', 'tanh', 'sigmoid'])
```

Exercise 1 – Solution (2/2)

```
# (d) Continuous, spans 4 orders of magnitude
    -> log-uniform.
Real(1e-5, 1e-1, prior='log-uniform')

# (e) Discrete count, narrow range -> Integer
    (uniform).
Integer(2, 30)
```

- Use `'log-uniform'` whenever the range spans multiple orders of magnitude and relative differences between values matter more than absolute ones.

Exercise 2 – Write a BayesSearchCV Call

Write a complete `BayesSearchCV` call to optimise a `RandomForestClassifier` on a dataset `(X, y)` with the following search space:

- `n_estimators`: integer from 10 to 300.
- `max_depth`: integer from 2 to 20.
- `min_samples_split`: integer from 2 to 20.
- `criterion`: `'gini'` or `'entropy'`.

Use 40 Bayesian iterations, 5-fold cross-validation, `'accuracy'` scoring, and random state 0. After fitting, print the best hyperparameter combination and the corresponding cross-validation score.

Exercise 2 – Solution (1/2)

Define the search spaces and imports:

```
from skopt import BayesSearchCV
from skopt.space import Integer, Categorical
from sklearn.ensemble import
    RandomForestClassifier

search_spaces = {
    'n_estimators':      Integer(10, 300),
    'max_depth':         Integer(2, 20),
    'min_samples_split': Integer(2, 20),
    'criterion':         Categorical(['gini',
        'entropy'])
}
```

Exercise 2 – Solution (2/2)

Create the optimiser, fit it, and print the results:

```
opt = BayesSearchCV(
    estimator=RandomForestClassifier(),
    search_spaces=search_spaces,
    n_iter=40,
    cv=5,
    scoring='accuracy',
    random_state=0
)
opt.fit(X, y)
print(opt.best_params_)
print(f"Best accuracy: {opt.best_score_:.4f}")
```

Exercise 3 – Debug the Code (1/2)

The following `BayesSearchCV` call contains **three bugs**. Find and fix all of them.

```
from skopt import BayesSearchCV
from skopt.space import Real
from sklearn.svm import SVC
from sklearn.datasets import load_breast_cancer

X, y = load_breast_cancer(return_X_y=True)
```

Exercise 3 – Debug the Code (2/2)

```
opt = BayesSearchCV(
    estimator=SVC, # line A
    search_spaces={
        'C':     Real(1e2, 1e-2,
                  prior='log-uniform'), # line B
        'gamma': Real(1e-4, 1e-1,
                  prior='log-uniform'),
    },
    n_iter=20,
    cv=5,
    random_state=42
)
opt.fit(X, y)
print(opt.best_score) # line C
```

Exercise 3 – Solution (1/2)

1. **Line A**: `SVC` is a class, not an instance. The `estimator` argument requires an instantiated object:

```
estimator=SVC()
```

2. **Line B**: `Real(1e2, 1e-2, ...)` has $low > high$ ($100 > 0.01$), which is invalid. The bounds must satisfy $low \leq high$:

```
'C': Real(1e-2, 1e2, prior='log-uniform')
```

Exercise 3 – Solution (2/2)

3. **Line C**: `best_score` is not a valid attribute. All fitted attributes in scikit-learn end with a trailing underscore:

```
1 print(opt.best_score_)
```

- As a general rule, any attribute computed during `fit()` in scikit-learn ends with _ (e.g. `best_params_`, `best_score_`, `best_estimator_`, `cv_results_`).

Revision

In this presentation you learned about:

- Why Bayesian optimisation outperforms grid and random search for expensive objectives.
- How the surrogate model and acquisition function guide the search.
- The `BayesSearchCV` API, its key parameters, and its fitted attributes.
- Defining search spaces with `Real`, `Integer`, and `Categorical`.
- When and why to use the `'log-uniform'` prior.
- How `n_iter`, `n_jobs`, `cv`, `scoring`, and `random_state` control the optimisation.
- A single-parameter search: tuning C of `LogisticRegression`.
- A multi-parameter search: jointly tuning C, γ, and `kernel` of `SVC`.